Two Edge-based Texture Measures

1. edgeness per unit area

$$F_{edgeness} = |\{ p \mid gradient_magnitude(p) \geq threshold\}| / N$$

where N is the size of the unit area

2. edge magnitude and direction histograms

$$F_{magdir} = (H_{magnitude}, H_{direction})$$

where these are the normalized histograms of gradient magnitudes and gradient directions, respectively.

Local Binary **Pattern** Measure

- For each pixel p, create an 8-bit number $b_1\ b_2\ b_3\ b_4\ b_5\ b_6\ b_7\ b_8$, where $b_i = 0$ if neighbor i has value less than or equal to p's value and 1 otherwise.

- Represent the texture in the image (or a region) by the histogram of these numbers.

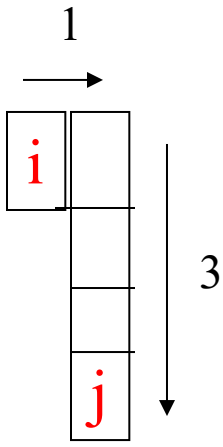
	1	2	3	
	100	101	103	
8	40	50	80	4
	50	60	90	5
	7	6		

→ 1 1 1 1 1 1 0 0

Co-occurrence Example

1	1	0	0
1	1	0	0
0	0	2	2
0	0	2	2
0	0	2	2
0	0	2	2

gray-tone image



$d = (3,1)$

$$C_d = \begin{array}{c|ccc} & 0 & 1 & 2 \\ \hline 0 & 1 & 0 & 3 \\ 1 & 2 & 0 & 2 \\ 2 & 0 & 0 & 1 \end{array}$$

co-occurrence matrix

From C_d we can compute N_d, the normalized co-occurrence matrix, where each value is divided by the sum of all the values.

The old-school way

Training

Training Images

Training Labels

Image Features → Training → Learned model

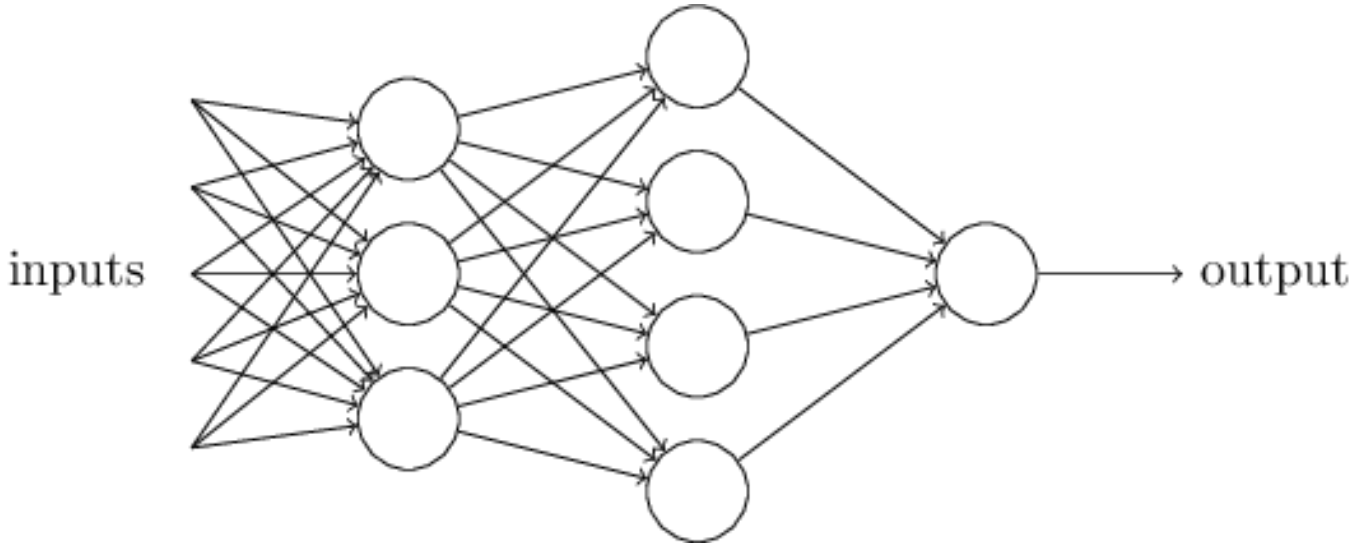
Testing

Test Image

Image Features → Learned model → Prediction

Slide credit: D. Hoiem and L. Lazebnik

Composition

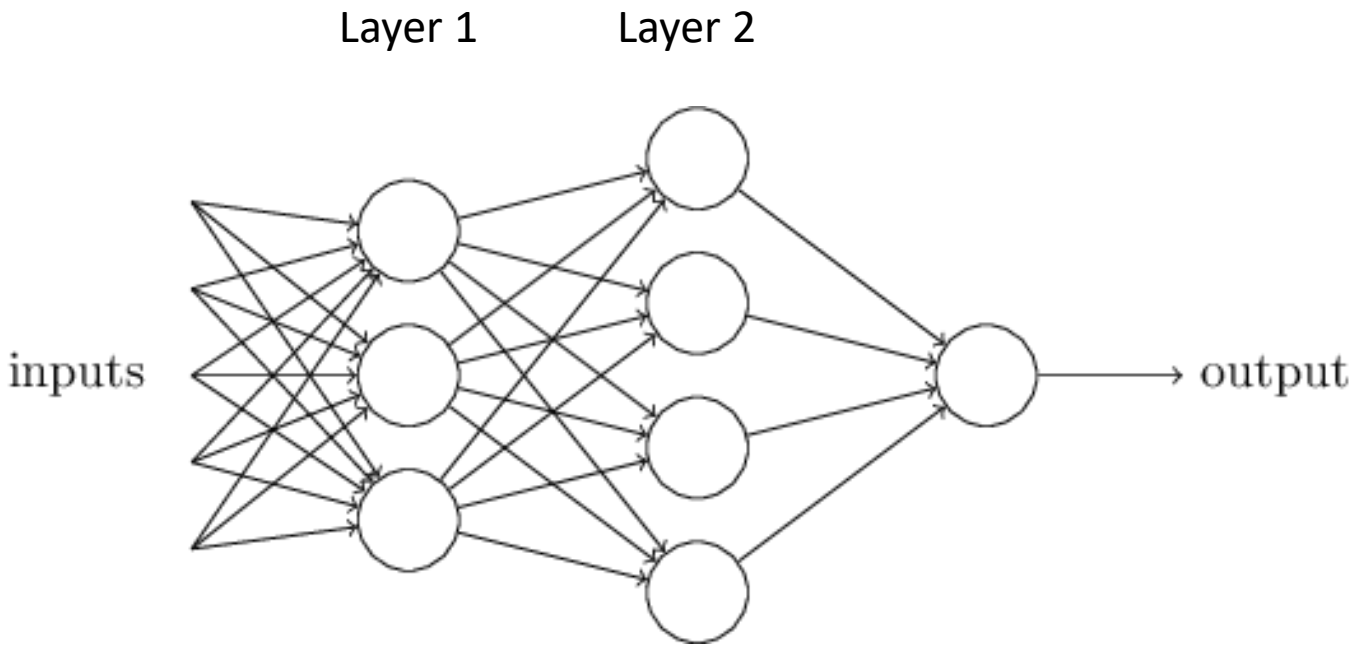


Attempt to represent complex functions as compositions of smaller functions.

Outputs from one perceptron are fed into inputs of another perceptron.

Nielsen

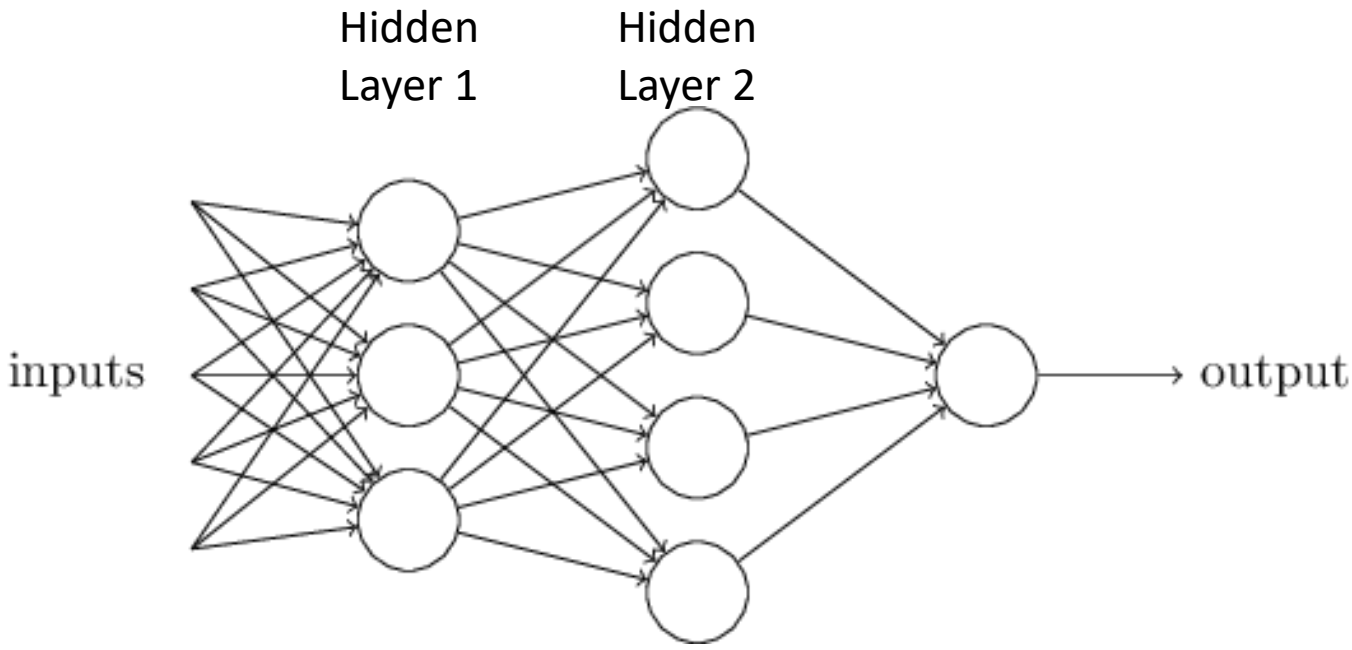
Composition



Sets of layers and the connections (weights) between them define the *network architecture.*

Nielsen

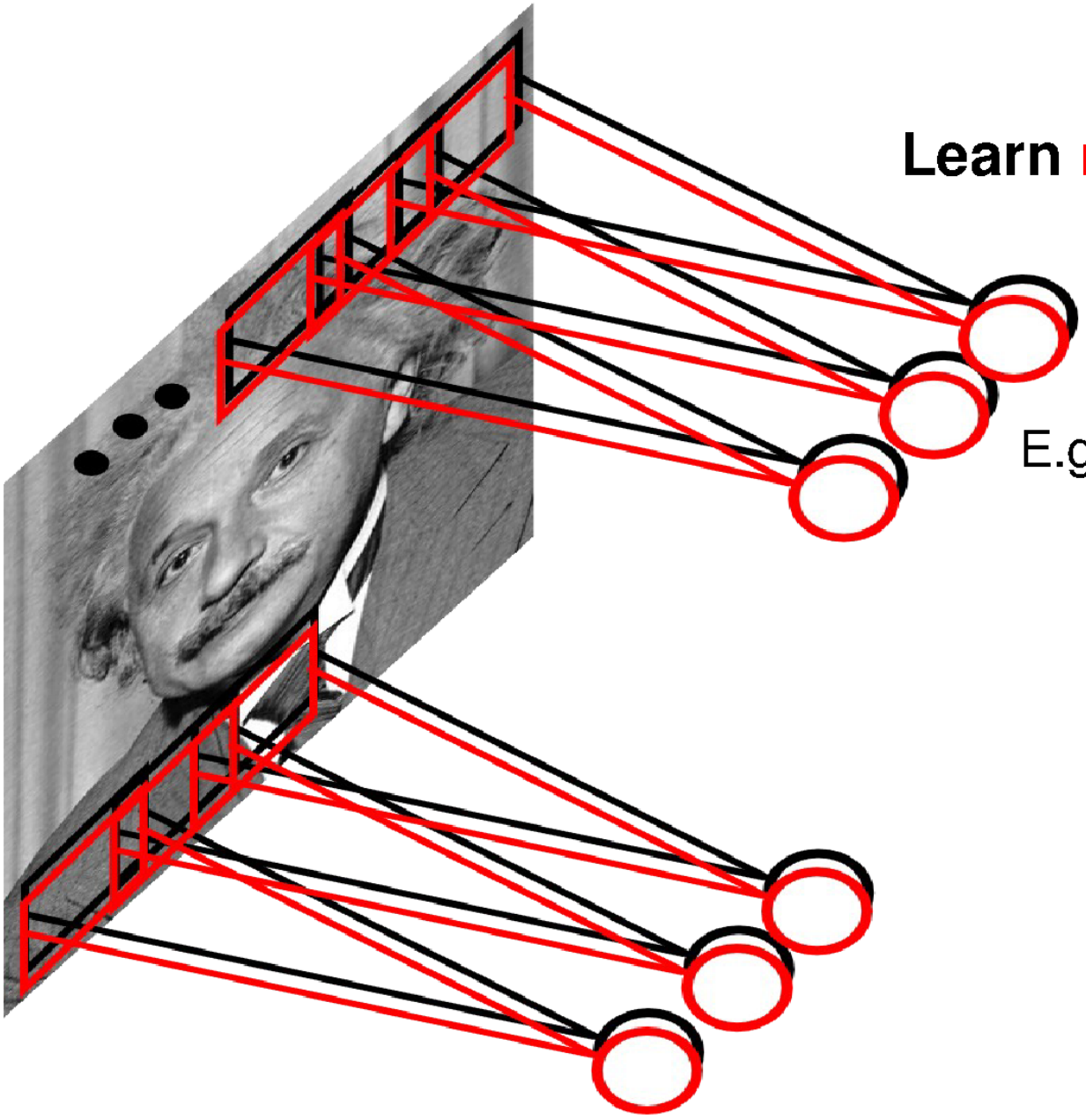
Composition



Layers that are in between the input and the output are called *hidden layers*, because we are going to *learn* their weights via an optimization process.

Nielsen

Convolutional Layer

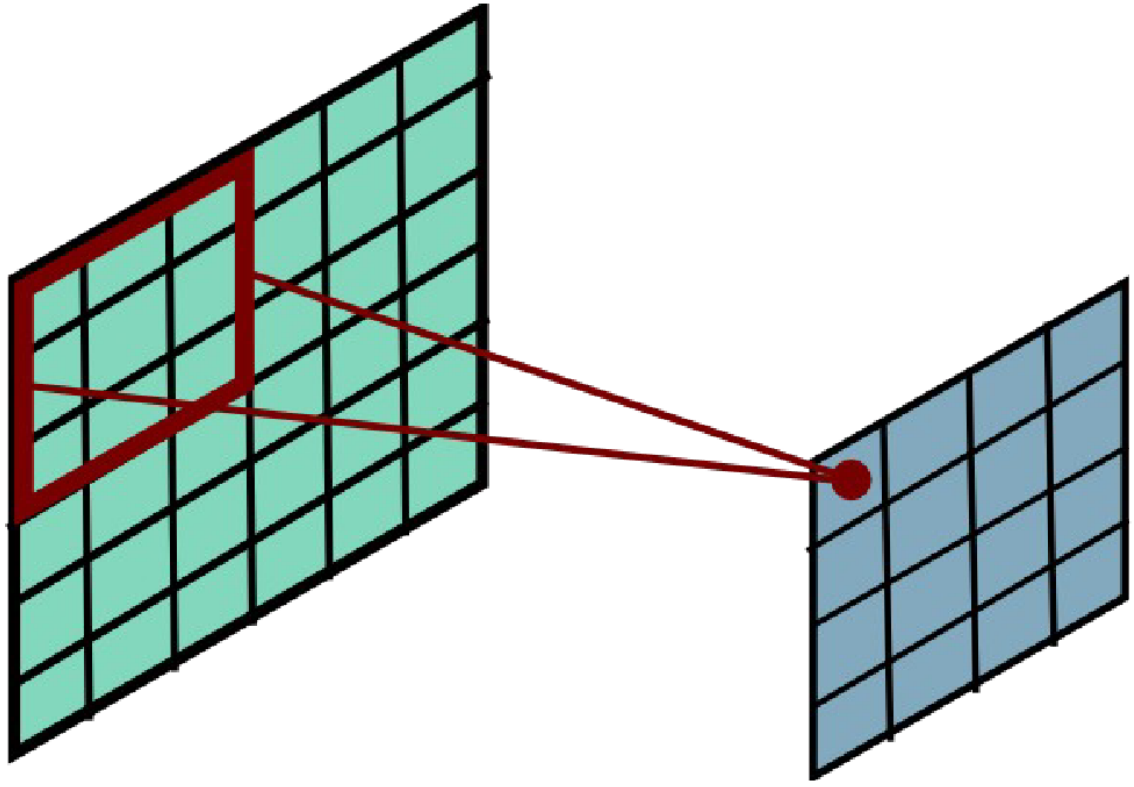


Learn multiple filters.

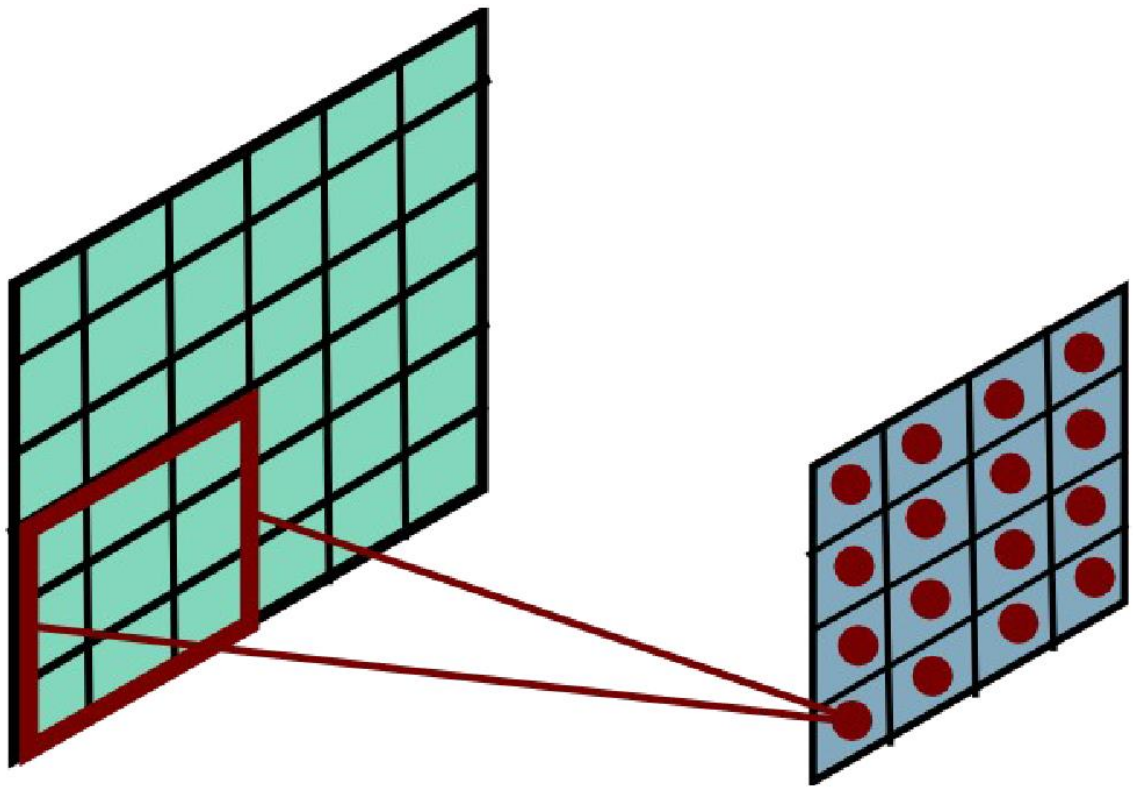
Filter = 'local' perceptron.
Also called *kernel.*

E.g.: 200x200 image
100 Filters
Filter size: 10x10
10K parameters

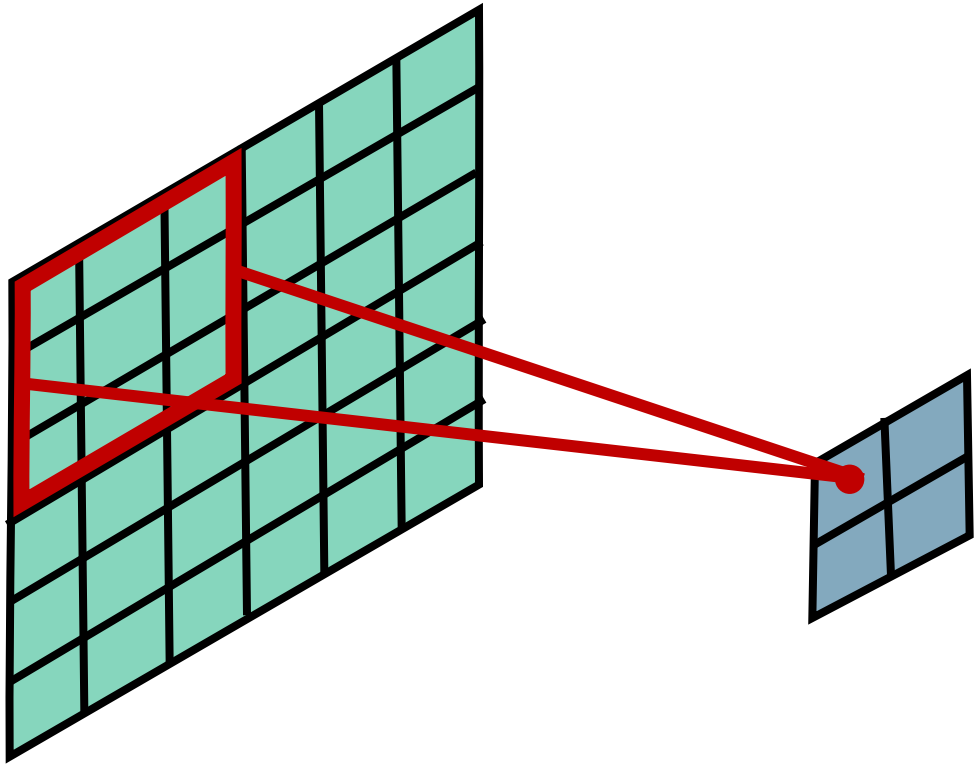
Stride = 1



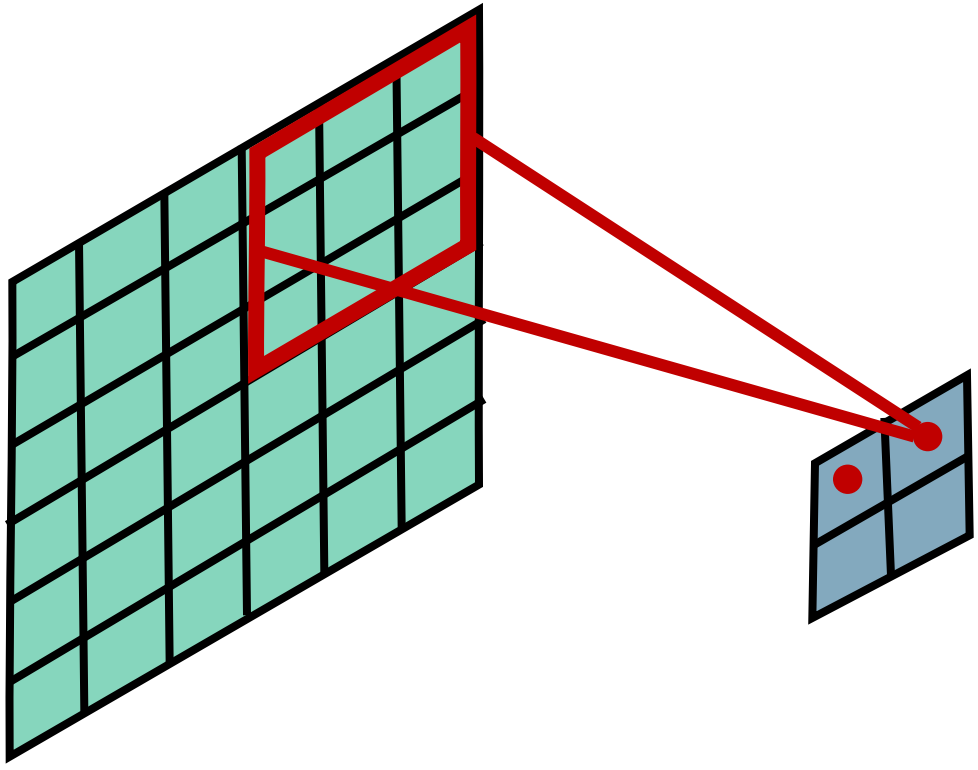
Stride = 1



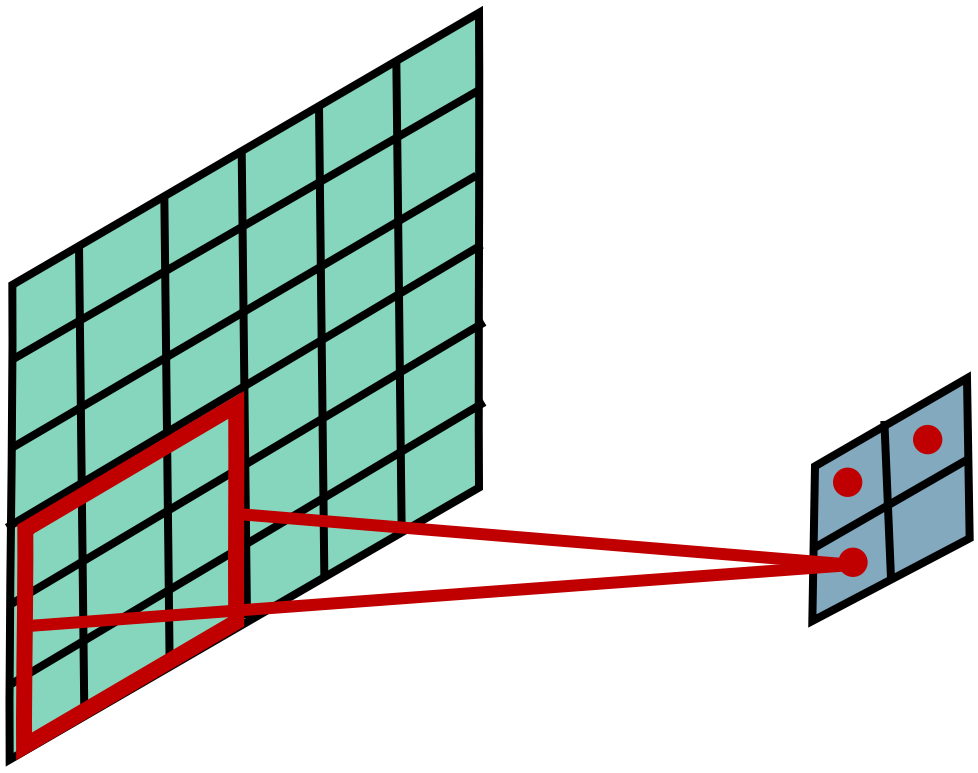
Stride = 3



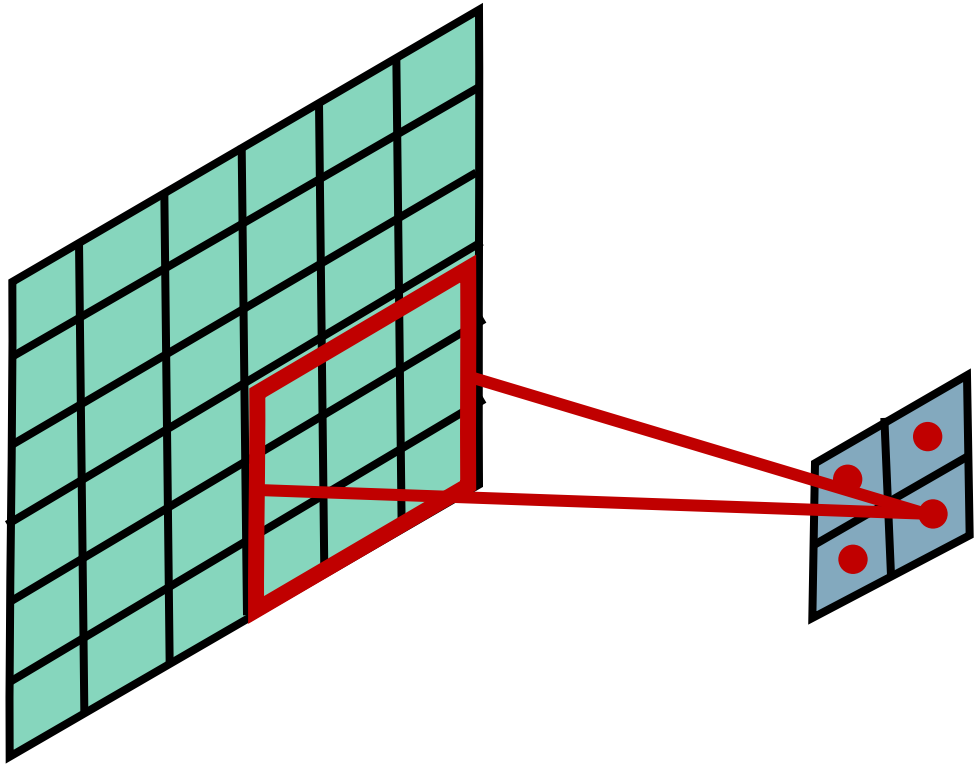
Stride = 3



Stride = 3

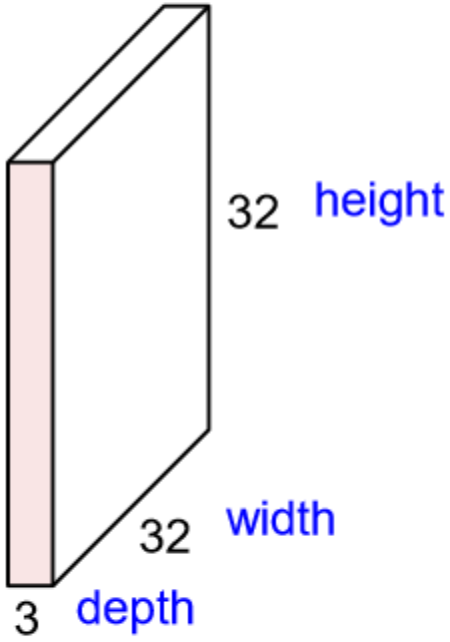


Stride = 3



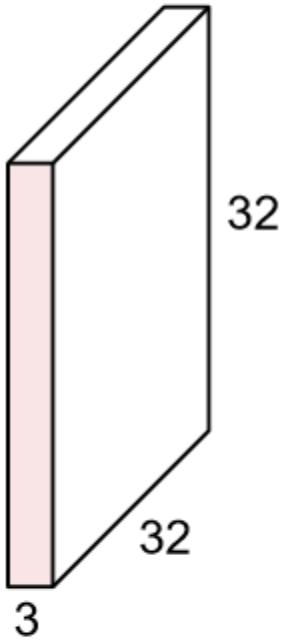
Convolutions: More detail

32x32x3 image



Convolutions: More detail

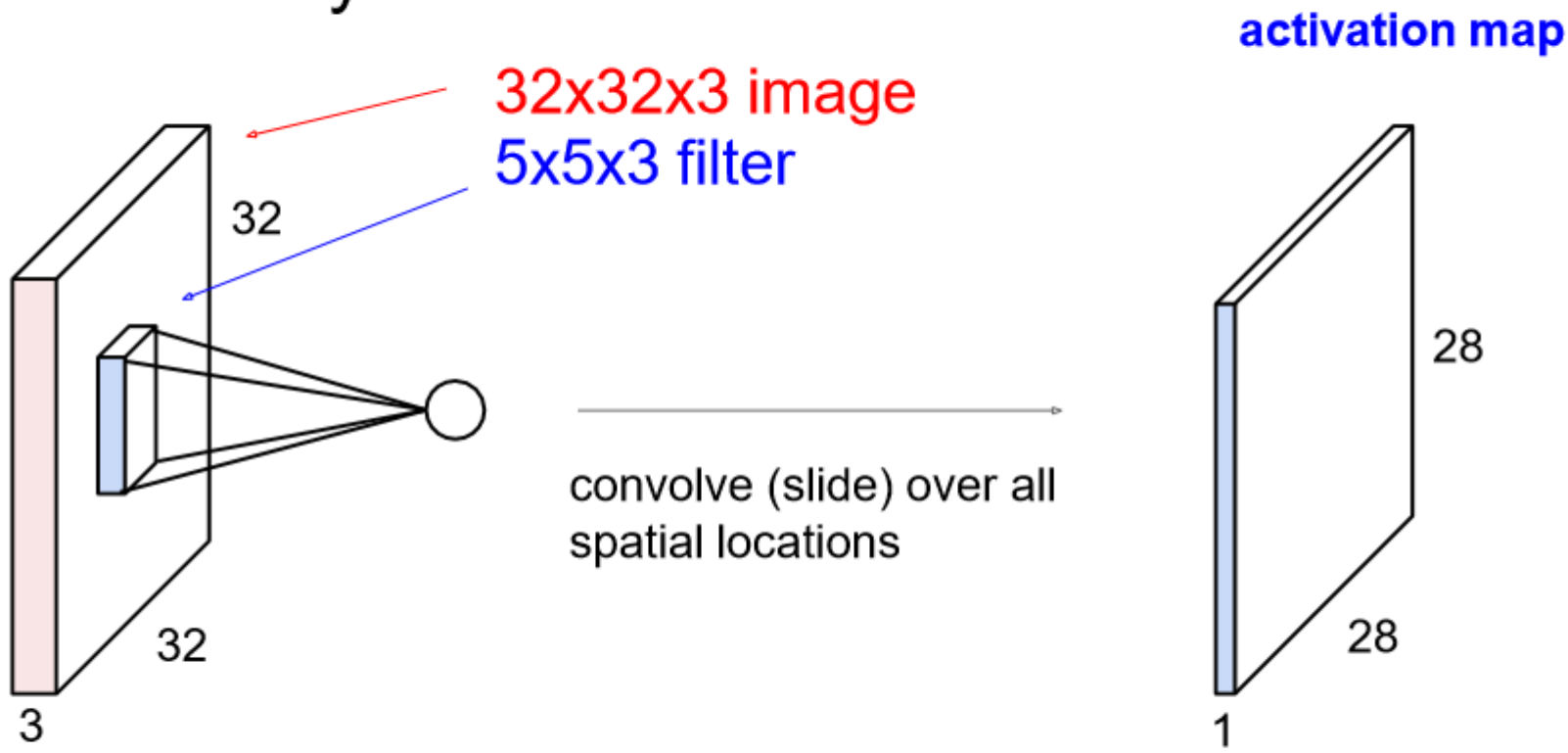
32x32x3 image



5x5x3 filter

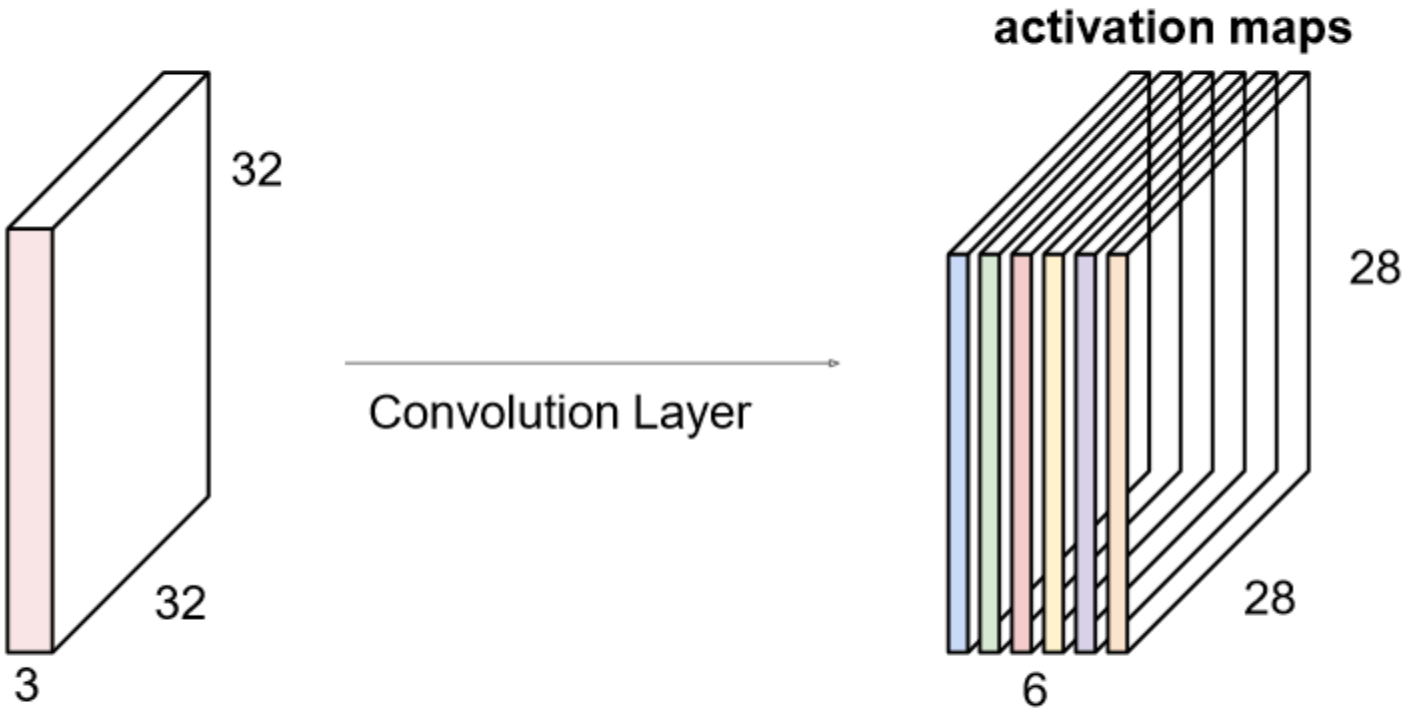
Convolutions: More detail

Convolution Layer



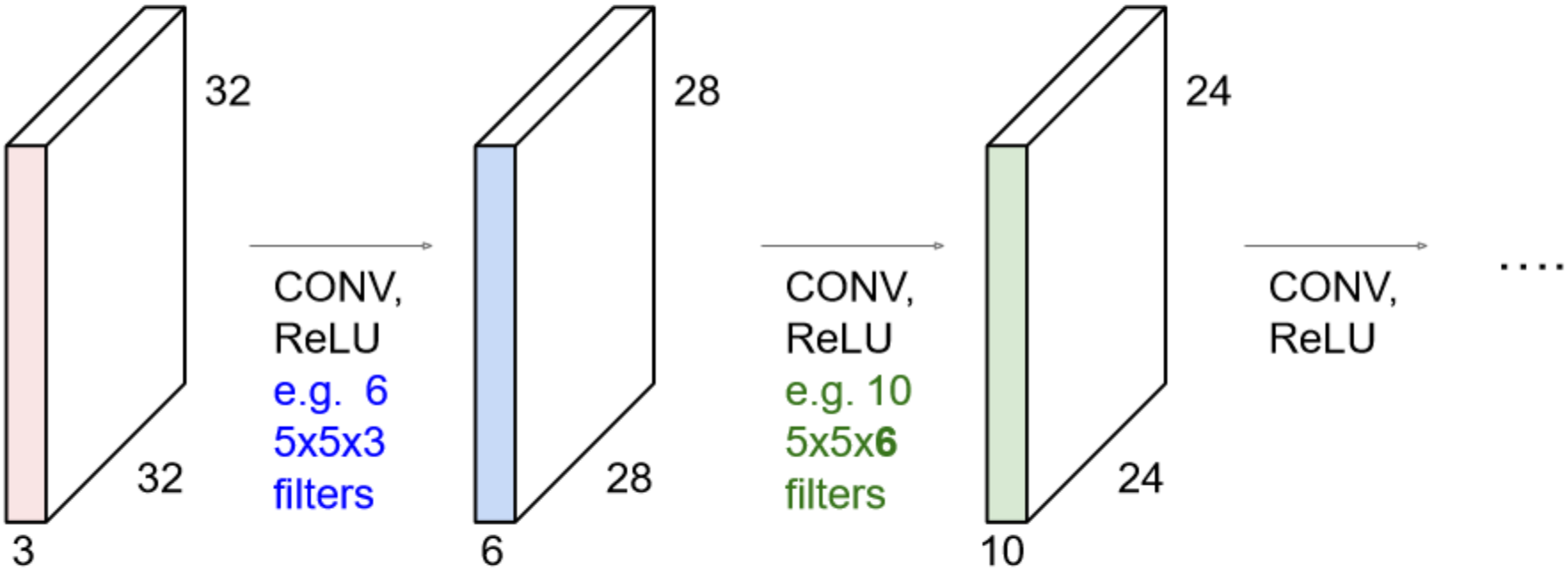
Convolutions: More detail

For example, if we had 6 5x5 filters, we'll get 6 separate activation maps:

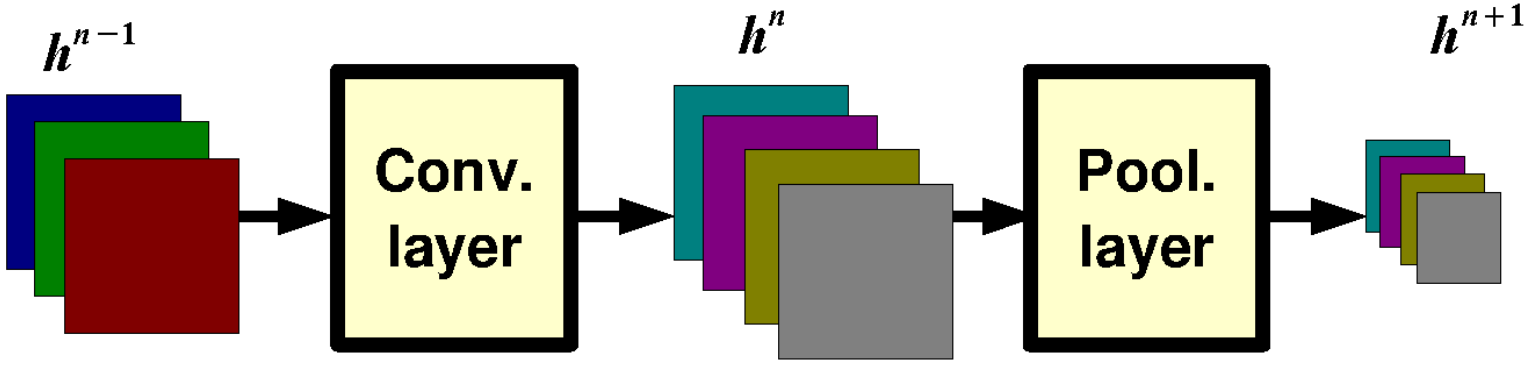


We stack these up to get a "new image" of size 28x28x6!

Convolutions: More detail



Pooling Layer: Receptive Field Size



Pooling Layer: Examples

Max-pooling:

$$h_j^n(x, y) = max_{\bar{x} \in N(x), \bar{y} \in N(y)} h_j^{n-1}(\bar{x}, \bar{y})$$

Average-pooling:

$$h_j^n(x, y) = 1/K \sum_{\bar{x} \in N(x), \bar{y} \in N(y)} h_j^{n-1}(\bar{x}, \bar{y})$$

Q1

- Compute the LBP code for the center pixel.
- If neighbor $\geq$ center output 1, otherwise output 0.
- **Bit order:** Start from **top-left**, then go **clockwise**:

60	55	40
30	50	80
45	50	20

Q1

- Compute the LBP code for the center pixel.
- If neighbor $\geq$ center output 1, otherwise output 0.
- **Bit order:** Start from **top-left**, then go **clockwise**:

	1	2	3	
	60	55	40	
8	30	50	80	4
	45	50	20	5
	7	6		

Neighbors vs 50: TL 60(1), T 55(1), TR 40(0), R 80(1), BR 20(0), B 50(1), BL 45(0), L 30(0)

Binary: **11010100**

Decimal: $= 2^7 + 2^6 + 2^4 + 2^2$

$= 128 + 64 + 16 + 4$

$= 212$

Q2

- Given image (values only 0, 1, 2):
- Build a co-occurrence matrix for a given offset.
- $d = (0, 1)$

0	1	1
2	1	0
2	2	0

Q2

- Given image (values only 0, 1, 2):
- Build a co-occurrence matrix for a given offset.
- $d = (0, 1)$

Step 1: List all valid pairs $(i, j)$from the image using this offset.

0	1	1
2	1	0
2	2	1

C(0,1)

C(1,1)

C(2,1)

C(1,0)

C(2,2)

C(2,1)

Q2

- Given image (values only 0, 1, 2):
- Build a co-occurrence matrix for a given offset.
- $d = (0, 1)$

0	1	1
2	1	0
2	2	1

Step 2: Fill the 3×3 co-occurrence matrix C for values $\{0, 1, 2\}$. Rows are i, columns are j.

C(0,1)

C(1,1)

C(2,1)

C(1,0)

C(2,2)

C(2,1)

$$\begin{array}{c|ccc} & 0 & 1 & 2 \\ \hline 0 & 0 & 1 & 0 \\ 1 & 1 & 1 & 0 \\ 2 & 0 & 2 & 1 \end{array}$$